

Project Management Web Application Proposal

Client Perspective

1. Executive Summary

As a rapidly growing Middleman contracting firm, we require a robust, flexible, and secure project management web application that supports our unique multi-tenant business model. This proposal outlines our requirements and expectations for a solution that empowers us to efficiently manage Clients, Experts, and Projects, while ensuring data isolation and operational control for each tenant. We seek a modern, scalable application leveraging Laravel 12 and Vue.js, designed with best practices for security, usability, and maintainability.

2. Project Goals & Objectives

- **Multi-Tenancy by Design:**

Each Middleman (tenant) operates in a fully isolated environment, accessible via a unique URL prefix (e.g., `yourapp.com/{tenant_slug}/...`).

- **Granular Role-Based Access:**

Clearly defined roles—Superadmin, Middleman, Expert, Client—each with tailored permissions, managed via Spatie's Roles & Permissions package.

- **End-to-End Project Lifecycle Management:**

Full suite of project management tools: creation, assignment, collaboration, file uploads, and status tracking.

- **Comprehensive Administration:**

Middlemen can onboard, edit, and monitor Clients and Experts within their own tenant scope.

- **Transparent Assignment & Budget Tracking:**

Assign Experts to projects, manage budgets, and monitor assignment statuses.

- **Insightful Reporting:**

Real-time dashboards and reports for profits, expenses, and KPIs—both tenant-specific and global (Superadmin view).

- **Enterprise-Grade Architecture:**

Built on Laravel 12 and Vue.js for reliability, security, and future scalability.

3. Functional Requirements

Module	Key Features
Tenant Management	Superadmin can create, suspend, and delete tenants (Middlemen); tenant settings for branding/features
Authentication & Roles	User registration/login, password reset, email verification, API authentication, role permissions
Project Management	Tenant-scoped project CRUD, file uploads, status workflow (Draft → Closed)
Client Management	Tenant-scoped client CRUD (id, name, email, phone, status)
Expert Management	Tenant-scoped expert CRUD (id, name, email, phone, status)
Assignment Module	Link Experts to Projects, manage budgets, assignment lifecycle
Reporting & Insights	Profit/expense tracking, dashboards, Vue.js chart components
Notifications	Email alerts for key events (project creation, assignments, deadlines)
Testing & Quality	Automated unit/feature tests, tenant isolation/security validation
Deployment & DevOps	Dockerized setup, CI/CD pipeline, enforced HTTPS, secure environment variables

4. Technical Architecture

- **Frontend:** Vue 3 SPA, Composition API, Pinia store, Vue Router
- **Backend:** Laravel 12 API, Sanctum authentication, Spatie Roles & Permissions
- **Database:** MySQL with tenant-specific foreign keys; global tenants and tenant_settings tables
- **Middleware:** Custom Laravel middleware for extracting {tenant_slug} and applying global tenant scope
- **Optional:** Real-time features via Pusher integration



5. Data Model Overview

1. **tenants** (id, slug, name, status, settings JSON)
2. **users** (id, tenant_id, name, email, password, role_id, status)
3. **projects** (id, tenant_id, name, type, start_date, deadline, budget_client, budget_expert, description, status)
4. **clients** (id, tenant_id, name, email, phone, status)
5. **experts** (id, tenant_id, name, email, phone, status)
6. **assignments** (id, tenant_id, project_id, expert_id, budget, status)
7. **expenses** (id, tenant_id, project_id, description, amount, date)
8. **files** (id, tenant_id, project_id, path, uploaded_by, uploaded_at)

7. Project Team

- Project Manager
- Backend Developer (Laravel)
- Frontend Developer (Vue.js)
- QA Engineer (Automated & Manual Testing)

8. Assumptions & Dependencies

- All content (Clients, Projects, Experts) provided by Middlemen
- Single currency for all monetary values
- Email via configured SMTP or transactional service
- Hosting supports VPS/Shared Hosting and Laravel requirements

9. Acceptance Criteria

- Strict tenant isolation—no data leakage across tenants
- Role permissions enforced for all user actions
- Project creation, assignment, and reporting as specified
- $\geq 80\%$ automated test coverage of core modules
- Successful end-to-end test on staging and production

10. Next Steps

1. Review and approve this proposal
2. Finalize additional feature requests or adjustments
3. Sign contract and schedule project kick-off meeting

Prepared for Genixstack Solutions

Date: 15th June 2025